clasificacion_transfer_learning

April 10, 2026

1 Proyecto: Clasificación de Imágenes con Transfer Learning

1.1 Objetivo

Clasificar imágenes usando un modelo preentrenado (ResNet) adaptado a nuestro dataset.

1.2 Dataset

CIFAR-10: 60,000 imágenes 32x32 en 10 clases.

---

[12]:
```
pip install torchvision
```

```
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-
packages (0.25.0+cu128)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from torchvision) (2.0.2)
Requirement already satisfied: torch==2.10.0 in /usr/local/lib/python3.12/dist-
packages (from torchvision) (2.10.0+cu128)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
/usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-
packages (from torch==2.10.0->torchvision) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
packages (from torch==2.10.0->torchvision) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-
packages (from torch==2.10.0->torchvision) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
(from torch==2.10.0->torchvision) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-
packages (from torch==2.10.0->torchvision) (2025.3.0)
Requirement already satisfied: cuda-bindings==12.9.4 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
```

```
(12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(11.7.3.90)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in
/usr/local/lib/python3.12/dist-packages (from torch==2.10.0->torchvision)
(1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-
packages (from torch==2.10.0->torchvision) (3.6.0)
```

```
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.12/dist-packages (from cuda-
bindings==12.9.4->torch==2.10.0->torchvision) (1.5.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from
sympy>=1.13.3->torch==2.10.0->torchvision) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from
jinja2->torch==2.10.0->torchvision) (3.0.3)
```

[13]:
```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
import matplotlib.pyplot as plt
import numpy as np
from tqdm import tqdm

# Configuración
BATCH_SIZE = 64
EPOCHS = 20
LR = 0.001
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

print(f"Usando: {DEVICE}")

# Semilla para reproducibilidad
torch.manual_seed(42)
```

```
Usando: cuda
```

[13]:
```
<torch._C.Generator at 0x79bb388414b0>
```

1.3 1. Preparar Datos

[14]:
```python
# Transforms
# CIFAR-10 es 32x32, ResNet espera mínimo 224x224
# Opción 1: Resize a 224x224 (más lento pero mejor)
# Opción 2: Adaptar ResNet para 32x32 (más rápido)

transform_train = transforms.Compose([
    transforms.Resize(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])  #␣
 ↪ImageNet stats
```

```
])

transform_test = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# Descargar datos
train_data = datasets.CIFAR10('./data', train=True, download=True,␣
 ↪transform=transform_train)
test_data = datasets.CIFAR10('./data', train=False, download=True,␣
 ↪transform=transform_test)

# Split train/val
train_size = int(0.9 * len(train_data))
val_size = len(train_data) - train_size
train_data, val_data = torch.utils.data.random_split(train_data, [train_size,␣
 ↪val_size])

# DataLoaders
train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)

# Clases
CLASSES = ['avión', 'auto', 'pájaro', 'gato', 'ciervo',
           'perro', 'rana', 'caballo', 'barco', 'camión']

print(f"Train: {len(train_data)}")
print(f"Val: {len(val_data)}")
print(f"Test: {len(test_data)}")
```

```
Train: 45000
Val: 5000
Test: 10000
```

1.4 2. Crear Modelo con Transfer Learning

[15]:
```
# Cargar ResNet18 preentrenado
modelo = models.resnet18(weights='IMAGENET1K_V1')

# Congelar capas del backbone
for param in modelo.parameters():
    param.requires_grad = False

# Descongelar últimas capas (fine-tuning parcial)
```

```
for param in modelo.layer4.parameters():
    param.requires_grad = True

# Reemplazar clasificador
num_features = modelo.fc.in_features
modelo.fc = nn.Sequential(
    nn.Linear(num_features, 256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 10)
)

modelo = modelo.to(DEVICE)

# Contar parámetros
total = sum(p.numel() for p in modelo.parameters())
trainable = sum(p.numel() for p in modelo.parameters() if p.requires_grad)
print(f"Parámetros totales: {total:,}")
print(f"Parámetros entrenables: {trainable:,} ({100*trainable/total:.1f}%)")
```

```
Parámetros totales: 11,310,410
Parámetros entrenables: 8,527,626 (75.4%)
```

1.5 3. Configurar Entrenamiento

[16]:
```
# Loss y optimizador
loss_fn = nn.CrossEntropyLoss()

# Learning rates diferentes para backbone y clasificador
optimizer = optim.Adam([
    {'params': modelo.layer4.parameters(), 'lr': LR * 0.1},
    {'params': modelo.fc.parameters(), 'lr': LR}
])

scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
```

1.6 4. Entrenar

[17]:
```
def train_epoch(model, loader, loss_fn, optimizer, device):
    model.train()
    total_loss, correct, total = 0, 0, 0

    for x, y in loader:
        x, y = x.to(device), y.to(device)

        optimizer.zero_grad()
        out = model(x)
```

```
        loss = loss_fn(out, y)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x.size(0)
        correct += (out.argmax(1) == y).sum().item()
        total += y.size(0)

    return total_loss / total, 100 * correct / total

def evaluate(model, loader, loss_fn, device):
    model.eval()
    total_loss, correct, total = 0, 0, 0

    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            out = model(x)
            loss = loss_fn(out, y)

            total_loss += loss.item() * x.size(0)
            correct += (out.argmax(1) == y).sum().item()
            total += y.size(0)

    return total_loss / total, 100 * correct / total
```

[18]:
```
# Entrenamiento
historial = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
best_acc = 0

for epoch in range(EPOCHS):
    train_loss, train_acc = train_epoch(modelo, train_loader, loss_fn,␣
 ↪optimizer, DEVICE)
    val_loss, val_acc = evaluate(modelo, val_loader, loss_fn, DEVICE)
    scheduler.step()

    historial['train_loss'].append(train_loss)
    historial['val_loss'].append(val_loss)
    historial['train_acc'].append(train_acc)
    historial['val_acc'].append(val_acc)

    print(f"Epoch {epoch+1}/{EPOCHS}")
    print(f"  Train - Loss: {train_loss:.4f}, Acc: {train_acc:.2f}%")
    print(f"  Val   - Loss: {val_loss:.4f}, Acc: {val_acc:.2f}%")

    # Guardar mejor modelo
    if val_acc > best_acc:
```

```
        best_acc = val_acc
        torch.save(modelo.state_dict(), 'best_model.pth')
        print(f"   Nuevo mejor modelo guardado!")
```

```
Epoch 1/20
  Train - Loss: 0.4969, Acc: 83.04%
  Val   - Loss: 0.3070, Acc: 89.28%
    Nuevo mejor modelo guardado!
Epoch 2/20
  Train - Loss: 0.2918, Acc: 90.23%
  Val   - Loss: 0.2714, Acc: 90.62%
    Nuevo mejor modelo guardado!
Epoch 3/20
  Train - Loss: 0.2329, Acc: 92.19%
  Val   - Loss: 0.2704, Acc: 90.66%
    Nuevo mejor modelo guardado!
Epoch 4/20
  Train - Loss: 0.1881, Acc: 93.52%
  Val   - Loss: 0.2333, Acc: 91.82%
    Nuevo mejor modelo guardado!
Epoch 5/20
  Train - Loss: 0.1584, Acc: 94.59%
  Val   - Loss: 0.2449, Acc: 92.14%
    Nuevo mejor modelo guardado!
Epoch 6/20
  Train - Loss: 0.1312, Acc: 95.61%
  Val   - Loss: 0.2458, Acc: 91.94%
Epoch 7/20
  Train - Loss: 0.1119, Acc: 96.27%
  Val   - Loss: 0.2407, Acc: 92.78%
    Nuevo mejor modelo guardado!
Epoch 8/20
  Train - Loss: 0.0927, Acc: 96.97%
  Val   - Loss: 0.2527, Acc: 92.64%
Epoch 9/20
  Train - Loss: 0.0754, Acc: 97.48%
  Val   - Loss: 0.2794, Acc: 92.32%
Epoch 10/20
  Train - Loss: 0.0681, Acc: 97.65%
  Val   - Loss: 0.2756, Acc: 93.02%
    Nuevo mejor modelo guardado!
Epoch 11/20
  Train - Loss: 0.0524, Acc: 98.23%
  Val   - Loss: 0.2882, Acc: 92.44%
Epoch 12/20
  Train - Loss: 0.0465, Acc: 98.42%
  Val   - Loss: 0.2622, Acc: 93.20%
    Nuevo mejor modelo guardado!
```

```
Epoch 13/20
  Train - Loss: 0.0358, Acc: 98.80%
  Val   - Loss: 0.2545, Acc: 93.40%
    Nuevo mejor modelo guardado!
Epoch 14/20
  Train - Loss: 0.0298, Acc: 98.98%
  Val   - Loss: 0.2722, Acc: 93.50%
    Nuevo mejor modelo guardado!
Epoch 15/20
  Train - Loss: 0.0237, Acc: 99.19%
  Val   - Loss: 0.2896, Acc: 93.44%
Epoch 16/20
  Train - Loss: 0.0214, Acc: 99.25%
  Val   - Loss: 0.2653, Acc: 93.84%
    Nuevo mejor modelo guardado!
Epoch 17/20
  Train - Loss: 0.0186, Acc: 99.38%
  Val   - Loss: 0.2820, Acc: 93.54%
Epoch 18/20
  Train - Loss: 0.0168, Acc: 99.40%
  Val   - Loss: 0.2641, Acc: 93.66%
Epoch 19/20
  Train - Loss: 0.0145, Acc: 99.52%
  Val   - Loss: 0.2619, Acc: 93.92%
    Nuevo mejor modelo guardado!
Epoch 20/20
  Train - Loss: 0.0133, Acc: 99.59%
  Val   - Loss: 0.2742, Acc: 93.90%
```

1.7 5. Evaluar

[19]:
```python
# Cargar mejor modelo
modelo.load_state_dict(torch.load('best_model.pth'))

# Evaluar en test
test_loss, test_acc = evaluate(modelo, test_loader, loss_fn, DEVICE)
print(f"\nTest Accuracy: {test_acc:.2f}%")
```

```
Test Accuracy: 94.22%
```

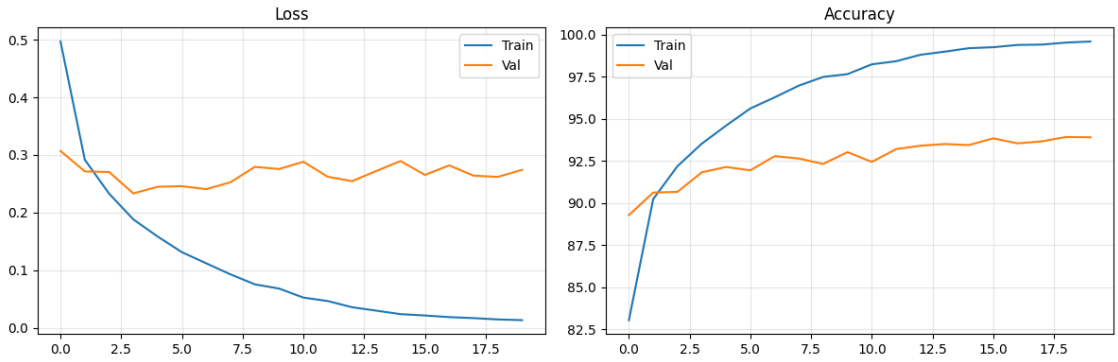
[20]:
```python
# Visualizar curvas
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(historial['train_loss'], label='Train')
axes[0].plot(historial['val_loss'], label='Val')
axes[0].set_title('Loss')
axes[0].legend()
```

```
axes[0].grid(True, alpha=0.3)

axes[1].plot(historial['train_acc'], label='Train')
axes[1].plot(historial['val_acc'], label='Val')
axes[1].set_title('Accuracy')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



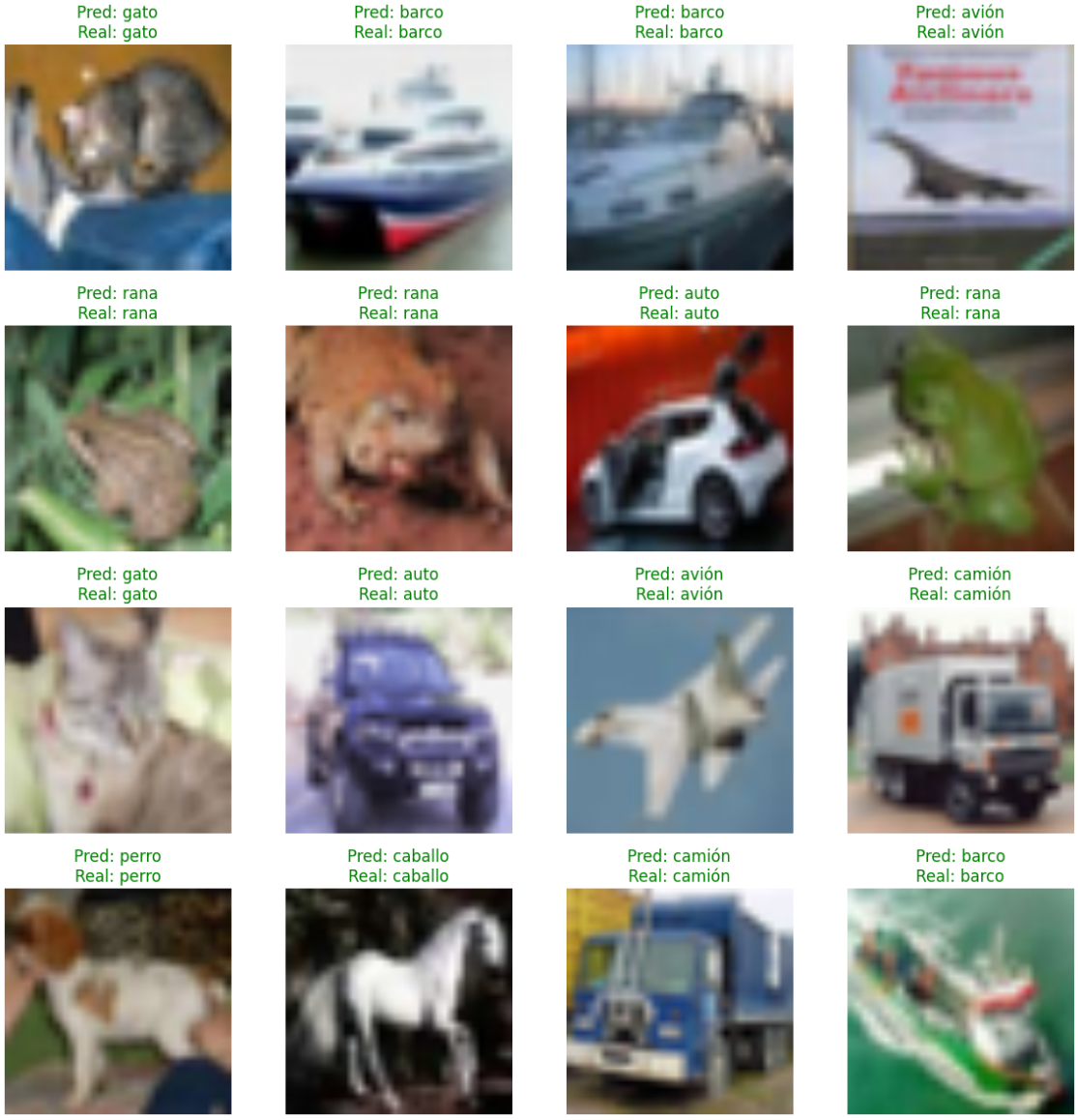
[21]:
```
# Visualizar predicciones
modelo.eval()
images, labels = next(iter(test_loader))
images, labels = images[:16].to(DEVICE), labels[:16]

with torch.no_grad():
    outputs = modelo(images)
    preds = outputs.argmax(1).cpu()

fig, axes = plt.subplots(4, 4, figsize=(12, 12))
for idx, ax in enumerate(axes.flatten()):
    img = images[idx].cpu().permute(1, 2, 0)
    img = img * torch.tensor([0.229, 0.224, 0.225]) + torch.tensor([0.485, 0.
 ↪456, 0.406])
    img = img.clamp(0, 1)

    ax.imshow(img)
    color = 'green' if preds[idx] == labels[idx] else 'red'
    ax.set_title(f"Pred: {CLASSES[preds[idx]]}\nReal: {CLASSES[labels[idx]]}",␣
 ↪color=color)
    ax.axis('off')
```

```
plt.tight_layout()
plt.show()
```



1.8 Conclusiones

- Transfer Learning permite alcanzar buenos resultados con pocos datos
- Fine-tuning parcial (solo últimas capas) es eficiente
- Learning rates diferentes para backbone y clasificador mejoran resultados